

Learning a function on a manifold

December 2025

1 Definitions

Definition 1. A metric tensor on (the embedded in $\mathbb{R}^k$) manifold $\mathcal{M}$ is a function $g : \mathcal{M} \rightarrow \mathbb{R}^{k \times k}, \forall x \in \mathcal{M}, \text{rank } g(x) = \dim \mathcal{M}, g(x) \succeq 0$.

Definition 2 (Dirichlet energy). The Dirichlet energy of a function $f : \mathcal{M} \rightarrow \mathbb{R}$ with respect to the manifold $\mathcal{M}$ equipped with the metric tensor g is defined as

$$\mathcal{E}_{\mathcal{M},g}(f) = \frac{1}{2} \int_{\mathcal{M}} \|\nabla_g f(x)\|_g^2 d\mu_g(x),$$

where $\nabla_g f$ denotes the gradient of f with respect to the metric g , $\|\cdot\|_g$ denotes the norm induced by g , and μ_g is the volume measure associated to g on $\mathcal{M}$.

Definition 3 (Measure). A Riemannian metric on a d -dimensional manifold is a smoothly varying inner product

$$g_x : T_x M \times T_x M \rightarrow \mathbb{R}.$$

In local coordinates, it is represented by a positive-definite matrix $g(x) = [g_{ij}(x)]$. The metric induces the Riemannian volume measure, often denoted:

$$d\mu_g(x) = \sqrt{\det g(x)} dx^1 \cdots dx^d.$$

This is the analogue of “area = $\sqrt{\det g}$ ” in differential geometry. $d\mu_g(x) = \sqrt{\det g(x)} dx^1 \cdots dx^d$.

Definition 4 (Gradient). The gradient of a function $f : \mathcal{M} \rightarrow \mathbb{R}$ with respect to the metric tensor g is defined as

$$\nabla_g f(x) = g(x) \nabla f(x) \in T_x \mathcal{M},$$

where $\nabla f(x)$ is the gradient of f with respect to the Euclidean metric.

Definition 5 (Laplacian). The Laplacian of a function $f : \mathcal{M} \rightarrow \mathbb{R}$ with respect to the metric g is defined as

$$\Delta_g f(x) = \nabla \cdot \nabla_g f(x) = \nabla \cdot (g \nabla f(x)) \in \mathbb{R}$$

2 Learning a function

Given:

- An embedded manifold $\mathcal{M} \subset \mathbb{R}^k, \dim \mathcal{M} = d$
- A metric tensor on $\mathcal{M}$: $g : \mathcal{M} \rightarrow \mathbb{R}^{k \times k}, \forall \mathbf{x} \in \mathcal{M}, \text{rank } g(\mathbf{x}) = d, g(\mathbf{x}) \succeq 0$.
Note: For simplicity, we assume $\det g(\mathbf{x}) = 1$ hence, the measure is the Lebesgue measure.
- A dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ where $\mathbf{x}_i \in \mathcal{M}, y_i \in \mathbb{R}$

Goal: to learn the smoothest function $f : \mathcal{M} \rightarrow \mathbb{R}$ such that the function $f(\mathbf{x}_i) \approx y_i$.

Constrained optimization:

$$\begin{aligned} & \min_f \mathcal{E}_{\mathcal{M},g}(f) \\ \text{s.t. } & f(\mathbf{x}_i) = y_i, \forall i \in [n]. \end{aligned}$$

Unconstrained optimization:

$$\begin{aligned} & \min_f \mathcal{E}_{\mathcal{M},g}(f) + \lambda \sum_{i=1}^n (f(\mathbf{x}_i) - y_i)^2 = \\ & \min_f \frac{1}{2} \int_{\mathcal{M}} \|\nabla_g f(\mathbf{x})\|_g^2 d\mu_g(\mathbf{x}) + \lambda \sum_{i=1}^n (f(\mathbf{x}_i) - y_i)^2 \end{aligned}$$

Gradient descent, convex optimization (in a *function* space):

$$\begin{aligned} \text{PDE: } \frac{\partial f}{\partial t} &= -\Delta_g f + 2\lambda \sum_{i=1}^n (f(\mathbf{x}_i) - y_i) \delta_{\mathbf{x}_i} \\ \text{step: } f_{t+1} &= f_t + \eta \left(\Delta_g f_t - 2\lambda \sum_{i=1}^n (f(\mathbf{x}_i) - y_i) \delta_{\mathbf{x}_i} \right) \end{aligned}$$

Definition 6 (Gradient-flow evolution operator). *We will denote $\Phi_T(f_i)$ as the gradient-flow evolution operator of f_i after time T of gradient descent. Note, that $\Phi_T(f_i) = \Phi_T(f_i, g, \mathcal{M}, D)$ depends on the metric tensor g , the manifold $\mathcal{M}$, and the dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$.*

Discretization in space (Disclaimer: this is a bit of a stretch.) If $\mathcal{M}$ is discretized as a point cloud $\mathcal{P} = \{\mathbf{z}_i\}_{i=1}^n$ with weights

$$w_{ij} = \exp \left(-\frac{\|\mathbf{z}_i - \mathbf{z}_j\|_{g(\mathbf{z}_i)}^2}{\epsilon} \right)$$

(epsilon TBD), one may argue that:

$$L_{\mathcal{P}} D_{\mathcal{P}}^{-1} \approx \Delta_g$$

Hence, one step of gradient descent becomes (yes, here comes the "stretch") like a transformer layer:

$$f_{t+1}(\mathbf{z}) = \underbrace{f_t(\mathbf{z})}_{\text{Residual}} + \eta \left(\underbrace{\frac{\sum_i \exp(-\|\mathbf{z} - \mathbf{z}_i\|_{g(\mathbf{z})}^2/\epsilon) f_t(\mathbf{z}_i)}{\sum_j \exp(-\|\mathbf{z} - \mathbf{z}_j\|_{g(\mathbf{z})}^2/\epsilon)}}_{\approx \text{Attention}} - \underbrace{2\lambda \sum_{i=1}^n (f(\mathbf{x}_i) - y_i) \delta_{\mathbf{x}_i}(\mathbf{z})}_{\text{Residual}} \right)$$

Idea 1: what if a fixed transformer performs gradient descent on the function space, hence, learns a function in context?

3 Learning a manifold

What if we are given a few smooth functions $f_1, \dots, f_l : \mathbb{R}^k \rightarrow \mathbb{R}$ on a manifold $\mathcal{M}$. However, we don't know the metric tensor g on $\mathcal{M}$. Can we learn the metric tensor such that the functions $f_1, \dots, f_l$ are as smooth as possible? If we do so, then we can easily learn a new smooth function from a few observations on the fly next time.

For this problem, the objective is the following:

$$\begin{aligned} \min_g \sum_{i=1}^l \mathcal{E}_{\mathcal{M},g}(f_i) \\ \text{s.t. } \det g(\mathbf{x}) = 1 \text{ for all } \mathbf{x} \in \mathcal{M}. \end{aligned}$$

where $\mathcal{E}_{\mathcal{M},g}(f_i)$ is the Dirichlet energy of f_i with respect to the metric tensor g . If we discard the constraint on the determinant or relax it to $\alpha I \preceq g(\mathbf{x}) \preceq \beta I$, the problem is convex w.r.t. g .

Idea 2: training a transformer-like model = learning a geometry of a manifold

Okay, as of now, we didn't have a need to perform GD in a function space inside the global optimization with respect to the metric tensor g – there was no need to train a transformer-like model with multiple layers. In what case would that be necessary?

4 More Complex Geometry

Okay, let us consider a more complex geometry so that performing gradient descent in a function space is necessary during optimization over g .

What if the metric tensor $g(x)$ also depends on the function $f(x)$ on the manifold: $g(x, f)$?

The Perona–Malik anisotropic diffusion Equation is given by

$$\frac{\partial u}{\partial t} = \nabla \cdot (c(|\nabla u|) \nabla u),$$

where $u(x, t)$ is the evolving signal or image, and the diffusivity function $c(\cdot)$ is typically chosen as

$$c(s) = \exp \left(-\left(\frac{s}{K} \right)^2 \right) \quad \text{or} \quad c(s) = \frac{1}{1 + \left(\frac{s}{K} \right)^2}.$$

Relevant literature

- Geshkovski et al. [Ges+25] – models transformers as interacting particle systems
- Belkin and Niyogi [BN06] – on the convergence of discrete Laplacian to Laplace-Beltrami operator
- Zhang and Zhou [ZZ25] – PDE view on transformers

References

- [BN06] Mikhail Belkin and Partha Niyogi. “Convergence of Laplacian Eigenmaps”. In: *Advances in Neural Information Processing Systems*. Vol. 19. MIT Press, 2006. (Visited on 12/05/2025).
- [Ges+25] Borjan Geshkovski et al. *A Mathematical Perspective on Transformers*. Aug. 2025. DOI: [10.48550/arXiv.2312.10794](https://doi.org/10.48550/arXiv.2312.10794). arXiv: [2312.10794](https://arxiv.org/abs/2312.10794) [cs]. (Visited on 12/05/2025).
- [ZZ25] Yukun Zhang and Xueqing Zhou. *PDE-Transformer: A Continuous Dynamical Systems Approach to Sequence Modeling*. Oct. 2025. DOI: [10.48550/arXiv.2510.03272](https://doi.org/10.48550/arXiv.2510.03272). arXiv: [2510.03272](https://arxiv.org/abs/2510.03272) [cs]. (Visited on 12/05/2025).